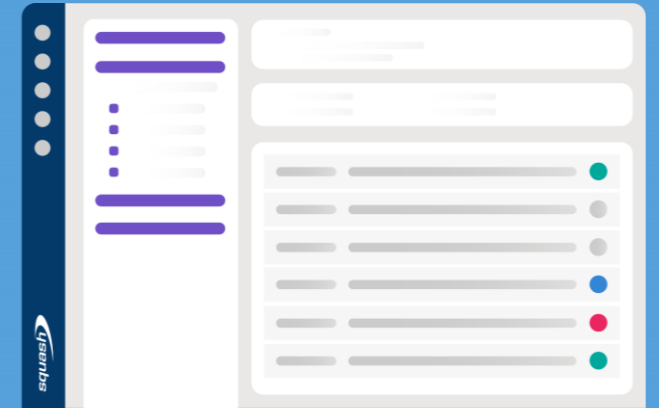


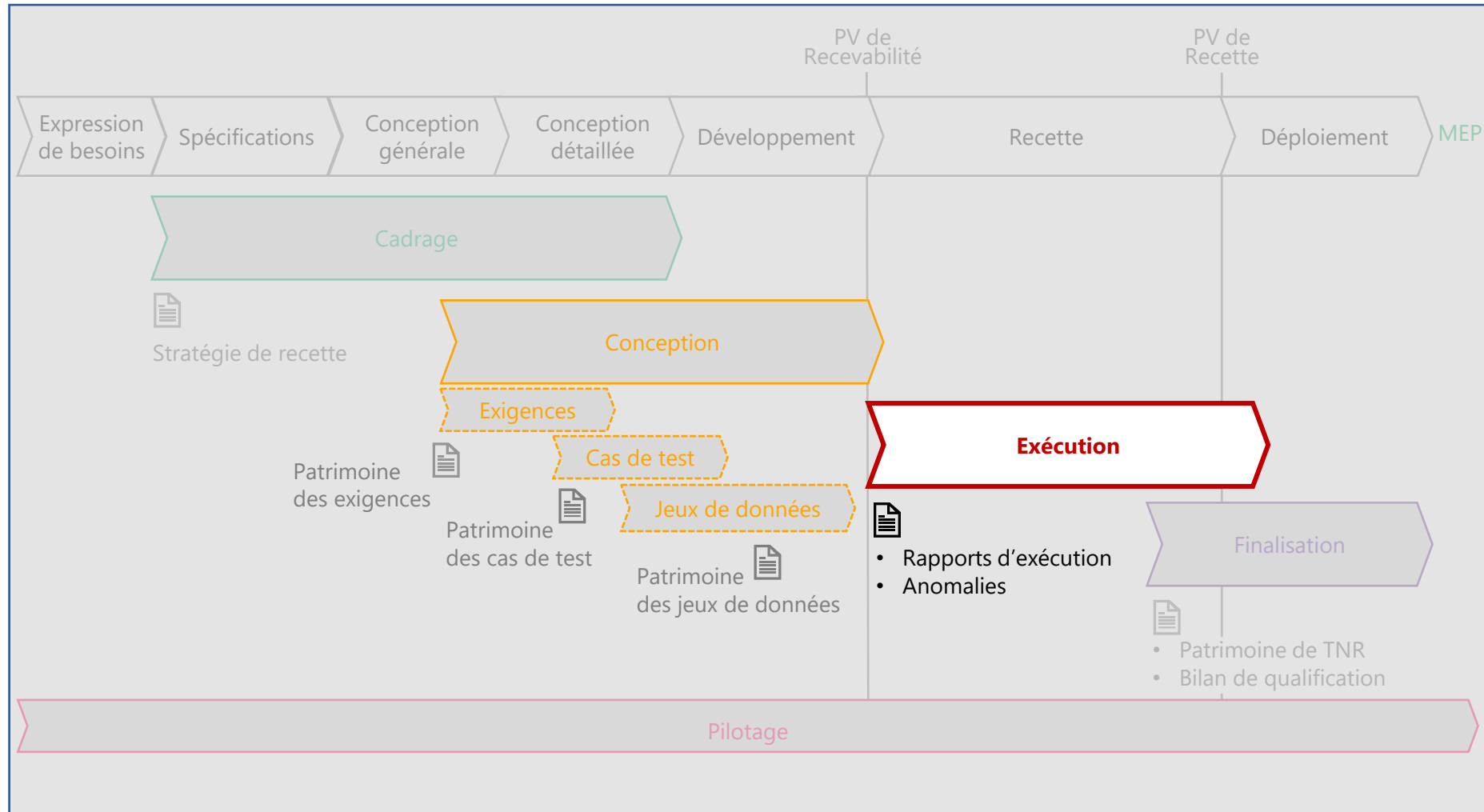
Les fondamentaux du test fonctionnel



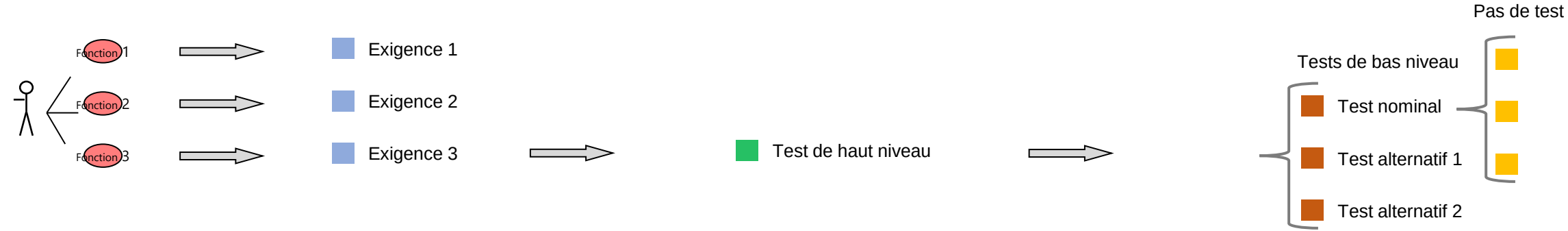
L'exécution des tests

1. Concevoir un plan d'exécution optimisé
2. Exécuter les tests
3. Comprendre la notion d'anomalie

La **vérification du niveau de Qualité** s'effectue durant la phase d'exécution, avec **l'exécution** des tests et la **remontée des anomalies**



L'exécution des tests
Récapitulatif



Fonctionnalité

Exigence

Test de haut niveau

Test nominal (détaillé/bas niveau)

Fonctionnalité	Exigence	Test de haut niveau	Test nominal (détaillé/bas niveau)	
Connexion à l'application	L'application doit permettre une connexion avec un login et un mot de passe valides	Étant donné que je suis sur la page de connexion Quand je rentre un login dans le champ « login » Et le mot de passe correspondant dans le champ « Mot de passe » Et que je clique sur le bouton [VALIDER] Alors j'arrive sur la page d'accueil	Actions	Résultats attendus
			Se connecter à la page https://...	La page de connexion s'affiche
			Saisir le login « Fabien » dans le champ login	Le login « Fabien » est affiché
			Saisir le mot de passe « 1234 » dans le champ mot de passe	Le champ mot de passe affiche « **** »
			Cliquer sur le bouton [VALIDER]	La page d'accueil du site est affichée

1. Concevoir un plan d'exécution optimisé



*« I love it when a plan comes together. »
(J'adore qu'un plan se déroule sans accroc.)*

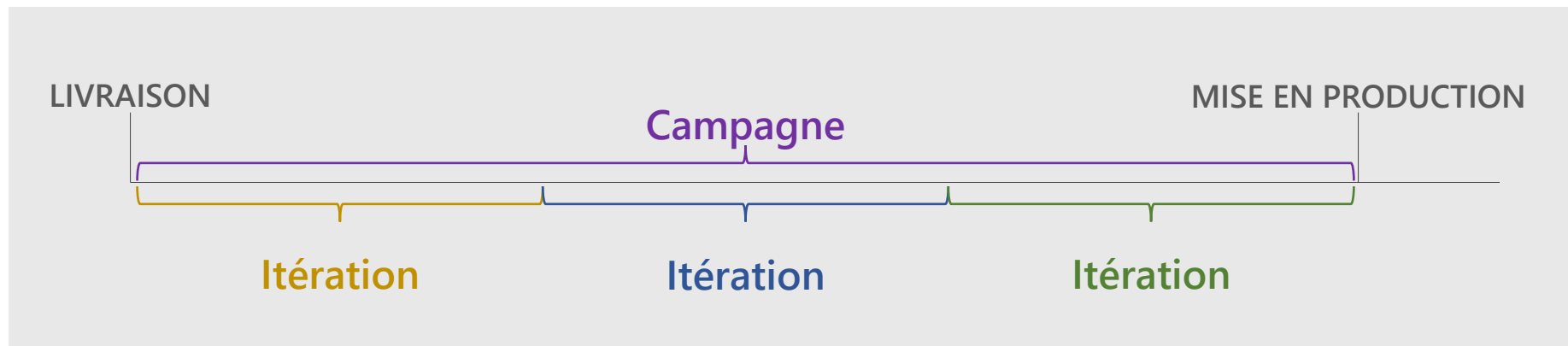
Colonel John « Hannibal » Smith –

Agence tous risques

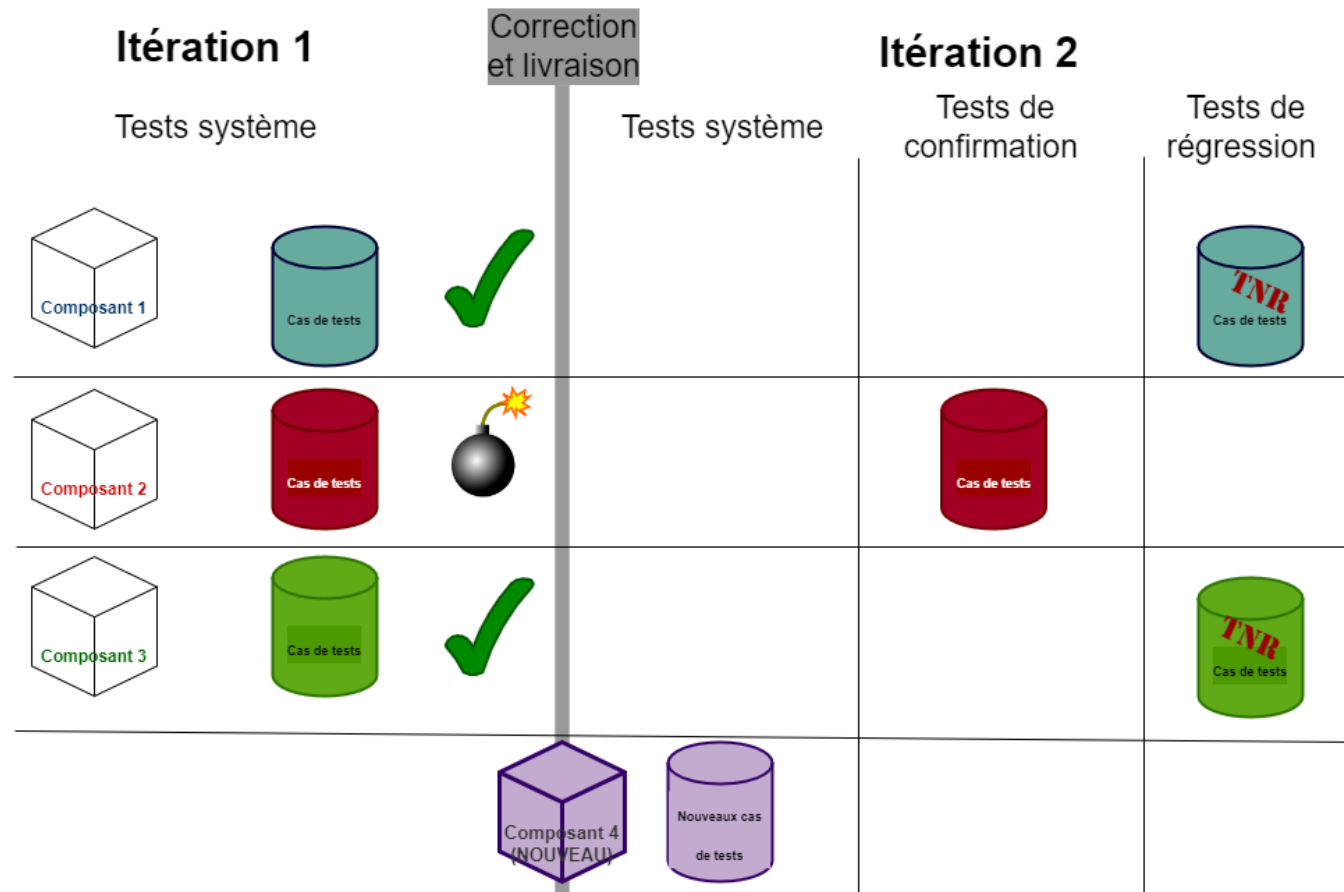
Une fois les exigences formalisées et reliées aux CT conçus pour assurer une couverture fonctionnelle du périmètre défini pour la recette de l'application, la **phase d'exécution** peut commencer.

Cette phase d'exécution ou « **campagne** » dans Squash TM est régie par certains principes :

- Une campagne **démarre** lors de la **première livraison de développement** et se **termine** au moment de la **mise en production (MEP)**.
- Une campagne (ou phase d'exécution) est **découpée en itérations** (ou cycles).
- Chaque itération est **planifiée entre deux livraisons de développement** (évolutions et/ou corrections).



- Exécuter une campagne :
 - Équivalent à une itération (*release N*)



- Concevoir une campagne de test c'est :
Sélectionner et organiser les scénarios que l'on va exécuter.
- Seuls 5% des campagnes de test concernent une **nouvelle** application « from scratch ».
- 95% du travail de qualité logicielle concerne des recettes **liées aux changements ou à la maintenance d'applications** :
 - Maintenance **évolutive**
 - Maintenance **corrective**
 - Campagnes de **Tests de Maintenance**

Rappel : On effectue généralement des **tests de maintenance** lorsque le SUT n'a pas été modifié mais que des modifications de son « éco-système » ont été réalisées.

- Deux notions de couverture fonctionnelle :

- En phase de Préparation :

La **couverture fonctionnelle des exigences par les tests** :

Proportion des exigences identifiées et validées couvertes par **au moins un cas de test**.

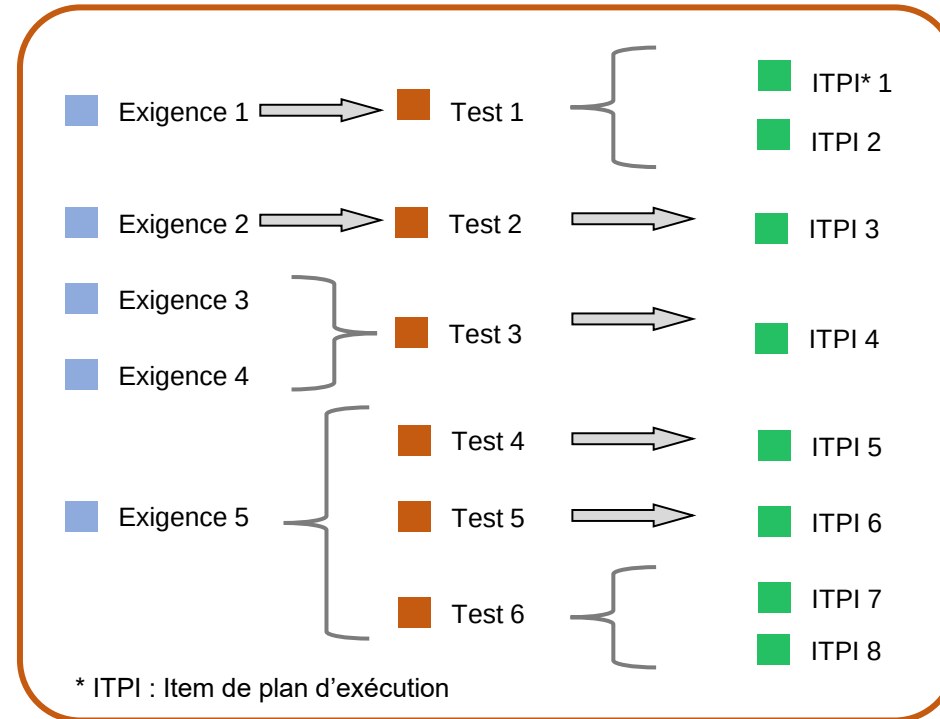
- En phase d'Exécution :

La **couverture fonctionnelle d'exécution** :

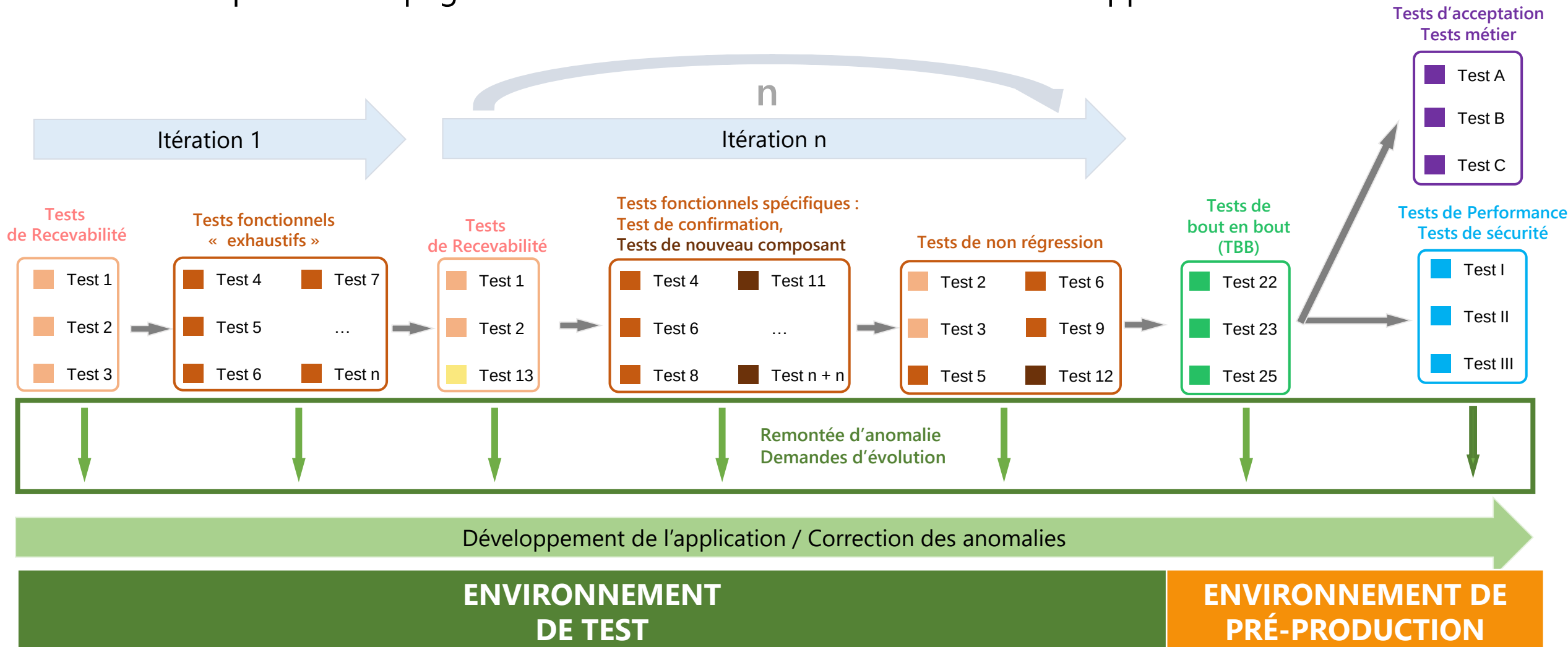
Proportion des exigences couvertes par les **tests exécutés dans la campagne**.

(Évolue en fonction de l'avancement)

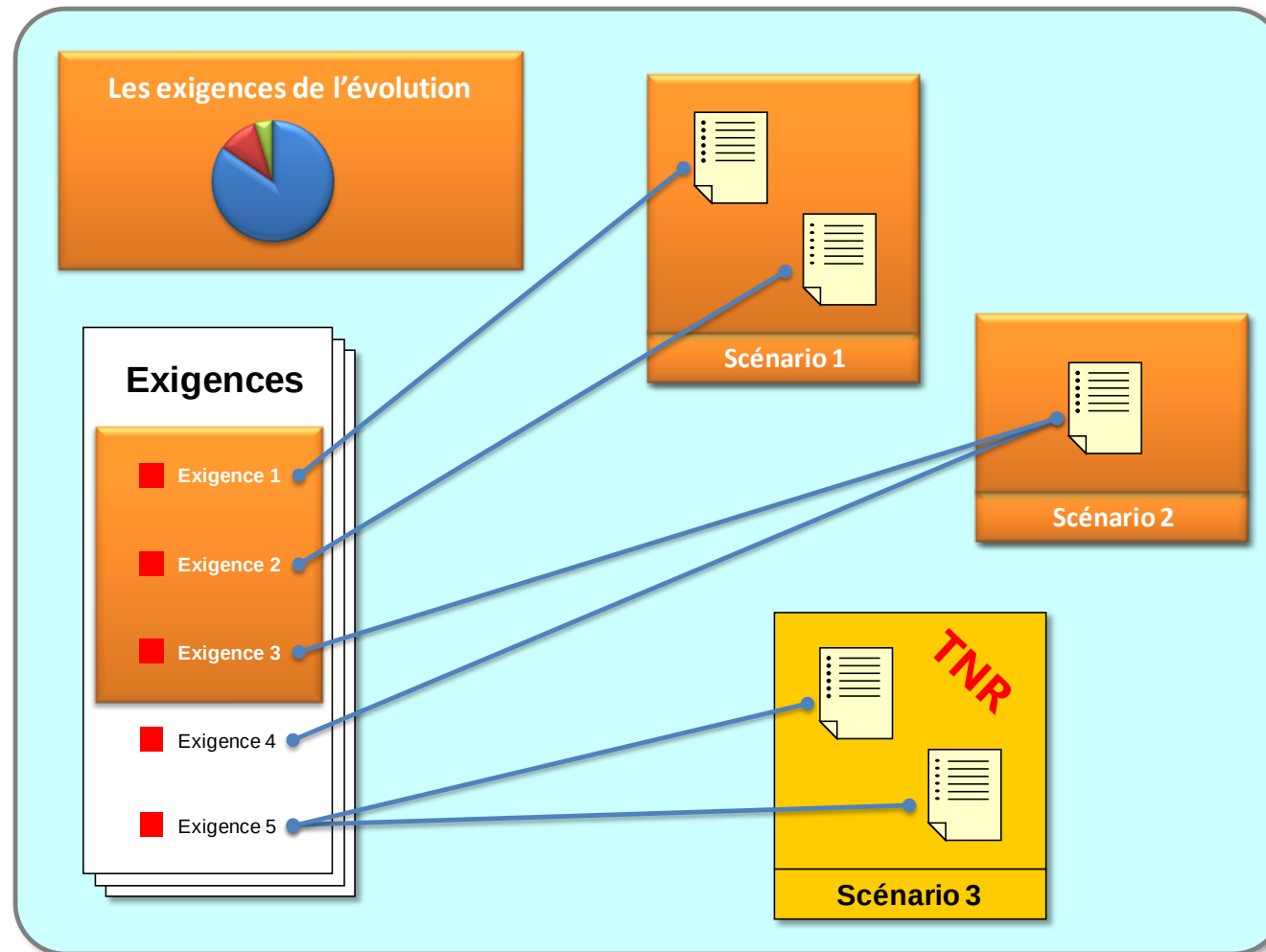
- Attention ! Cette notion est **différente** de celle rattachée à la **proportion de cas de tests exécutés par rapport à la quantité totale de cas de test** à exécuter, d'où découle le « Reste à faire », quantitatif.



- Schéma complet de campagne de recette fonctionnelle d'une **nouvelle** application :



- Plan d'exécution d'une campagne de recette fonctionnelle suite à une **évolution applicative** :



- Les différents types de campagne de tests :
 - **Campagne de recevabilité:**
Pour valider qu'une livraison est de qualité globale suffisante pour entrer en recette.
NB : Peut également être réalisée en tant qu'itération, ou de cas de test à réaliser à chaque livraison
 - **Campagne de tests fonctionnels** V1, V2, Vn :
Périmètre fonctionnel à définir.
 - **Campagne de TNR** (Tests de Non Régression) :
En fin d'itération.
 - **Campagne de TBB** (Tests de Bout en Bout) :
En fin de cycle/campagne.

- **Quand concevoir une campagne de test :**

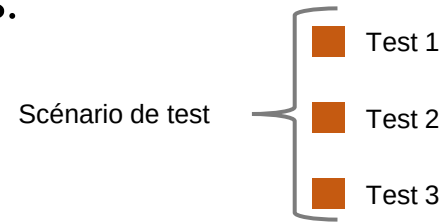
- La conception de l'organisation et de l'objectif des campagnes se fait en amont, au moment de la définition de la stratégie de recette

- **Les itérations dans une campagne :**

En règle générale, une recette complète comporte de 3 à 5 campagnes itératives (itérations) de test qui viennent tester les livraisons correspondantes de l'application.

- Dans le cadre de livraisons loties (= par lots incrémentaux), chaque lot fait l'objet de plusieurs itérations.

- Constituer des scénarios de tests consiste à **assembler** et **organiser** les **cas de tests** en **scénarios**, généralement orientés « processus métier ».
- **Un scénario de test** est le concept qui précise la fonction à tester. Un scénario de test est composé d'un ou plusieurs cas de tests.
- **Exemples :**



Scénario 1 : la gestion d'un utilisateur

- Test 1 : Créer un nouvel utilisateur
- Test 2 : Modifier un utilisateur
- Test 3 : Supprimer un utilisateur

Scénario 2 : le changement d'adresse

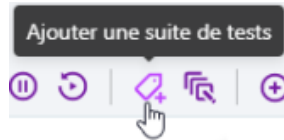
- Test 1 : Changement d'adresse en France
- Test 2 : Changement d'adresse en Europe
- Test 3 : Changement d'adresse aux USA

- Les scénarios de test sont constitués autour de regroupements fonctionnels de l'application :
 - **Composant de l'application :**
module de comptabilité / module d'impression des factures / module de gestion de trésorerie.
 - **Périmètre fonctionnel :**
fonctions de connexion / fonctions de création de client / fonctions d'édition des clients.
 - **Proximité d'exigences :**
respect des règles métier / exigences des interfaces / exigences des traitements.
- L'organisation des scénarios dépend :
 - **de la configuration** / architecture logicielle de l'application
 - **du niveau** de qualité visé
 - **de la performance** des testeurs / **du niveau d'organisation** du service Qualité

Les scénarios de test peuvent être créés et mis en forme sur Squash en utilisant la fonctionnalité de création de **suites de tests** permettant ainsi d'organiser l'exécution des tests au sein d'une itération.

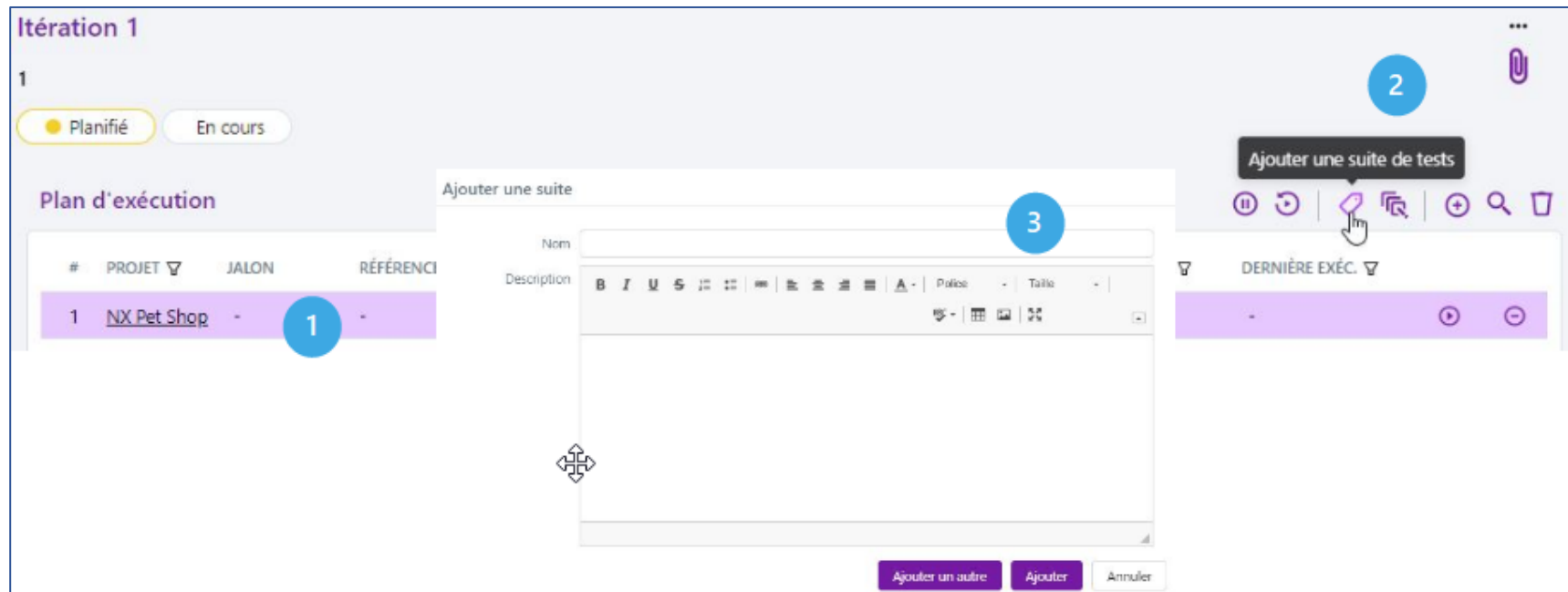
1 Sélectionner des cas de tests.

2 Cliquer sur



3

- Renseigner un libellé et cliquer sur **Ajouter** pour ajouter une nouvelle suite de tests
- Ou bien
- Affecter à une suite de tests existante.





Créer une campagne, une itération, des suites de test et le plan d'exécution

1. Créer une campagne « DAB » et renseigner la description et les dates du planning.
2. Ajouter des cas de test au plan d'exécution de la campagne.
3. Créer une itération et cocher l'option 'Copier le plan d'exécution de la campagne'.
4. Organiser le plan d'exécution en suites de tests en fonction des scénarios de test envisagés.
5. Assigner, filtrer et réorganiser vos plans d'exécutions.

3. Exécuter les tests

- Exécuter un test, c'est :
 - Lire la **description** du cas de test, comprendre son **objectif**;
 - Dérouler les **pas de test** prévus dans les tests;
 - Exécuter les **manipulations** décrites;
 - Observer le **comportement** de l'application;
 - Comparer ce comportement avec le **résultat attendu** :
 - Si le résultat obtenu correspond au résultat attendu → **OK**
 - Si le résultat obtenu est « divergent » → **KO**
 - Renseigner le résultat dans l'outil de gestion de recette.

Cas de test

Description :

Ce cas de test vérifie que l'accès à la page d'accueil est possible après identification avec un login existant et le mot de passe correspondant.

Actions

Se connecter à la page
https://...

Saisir le login « **Fabien** »
dans le champ login

Saisir le mot de passe
« **1234** » dans le champ
mot de passe

Cliquer sur le bouton
[VALIDER]

Résultats attendus

La page de connexion
s'affiche



Succès

Le login « **Fabien** » est
affiché



Succès

Le champ mot de passe
affiche « ******** »



Échec

La page d'accueil du site
est affichée



Succès

- La notion de **OK** :
 - Données constatées en sortie identiques aux données en sortie attendues
 - Comportement attendu
 - Application stable
 - Retour dans un état initial, en mode de fonctionnement optimal.
- La notion de **KO** :
 - Différence dans les données attendues
 - Divergence de comportement
 - Plantage total d'application
 - Fin de procédure dans un état autre que celui attendu dans la description du test

- Quand un test est **KO** :
 - Renseigner le test comme KO dans l'outil de gestion des tests (Squash TM).
 - Ouvrir un ticket d'anomalie dans l'outil de gestion des anomalies (p. ex. Mantis BugTracker)
 - Décrire précisément les divergences de comportement observé (IHM).
 - Décrire précisément les divergences de données (jeu de données).
 - Rajouter :
 - Copies d'écran
 - Données recueillies en sortie
 - Tout ce qui peut être utile à l'analyse de l'échec du test (navigateur, etc.)
 - **On « déclare une anomalie » dans l'outil de gestion des anomalies** (ex : *MANTIS BT*).

- Finaliser le test :
 - **Le test ne s'arrête pas si on trouve un « pas de test » KO.**
 - Il faut le continuer jusqu'au dernier pas de test et faire toutes les manipulations décrites.
- Comment faire si on est bloqué sur un « pas de test » ?
 - Un 'pas de test' comporte une manipulation qui empêche le comportement attendu de l'application => **C'est une anomalie à déclarer.**
 - Pour poursuivre le test, on doit le reprendre du début, mais **en évitant**, si possible, le 'pas de test' porteur de l'anomalie pour terminer l'ensemble de la procédure testée : il s'agit d'un **contournement**.

L'exécution des tests consiste à attribuer un statut d'exécution à **chaque pas de test, ou cas de test**.



À exécuter



En cours



Succès



Arbitré



Non testable



Bloqué



Échec

Chaque **suite de test** a un statut attribué en fonction du statut des tests de la suite.

Le statut attribué à une **suite de test** répond à un **ordre de priorité** donné à chacun des statuts.

Voici l'ordre de priorité des statuts principaux. Il suffit d'un seul test au statut le plus élevé pour que la suite de tests prenne le statut de celui-ci :

- « Bloqué »
- « Échec »
- « Succès » : un test en succès met la suite de test au statut « en cours »

Par exemple : pour cette suite, un seul cas de test est au statut « bloqué ». Peu importe le statut des autres tests, la suite aura le statut « Bloqué ».

Plan d'exécution

#	PROJET	JALON	RÉFÉRENCE	CAS DE TEST	IMP.	JDD	STATUT	%	UTILISATEUR
1	Open Ecole	-	ECOL_04	Suppression d'une éco...	🚩	-	🟡	42 %	Squash Adn
2	Open Ecole	-	ECOL_01	Accès au formulaire d...	🚩	FireFox	🟢	100 %	Squash Adn
3	Open Ecole	-	ECOL_01	Accès au formulaire d...	🚩	IE	⬜	0 %	-



Exécuter la campagne de test pour le DAB

- Exécuter les tests de 3 manières possibles

4. Comprendre la notion d'anomalie



« Le pire des défauts est de les ignorer. »

Publius Syrus



- Qu'est-ce qu'une anomalie ?

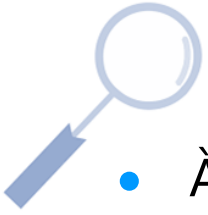
- Une anomalie c'est :
 - Anomalie / défaut / *defect** / dysfonctionnement / *bug** / non-conformité / inaptitude au bon fonctionnement.
 - Une anomalie est un **comportement** de l'application observé lors de l'exécution d'un test, qui ne correspond pas au comportement décrit dans les spécifications fonctionnelles.

* Terme anglais



- Qu'est-ce qu'une carence ?

- Une carence c'est :
 - Un **comportement** inattendu de l'application.
Les spécifications ne sont pas explicites et **ne décrivent pas le comportement attendu.**
 - Ce type de dysfonctionnement peut conduire à une **demande d'évolution de la MOA auprès de la MOE** et ainsi aboutir à **une nouvelle livraison** ou à une **solution de contournement.**



- À quoi sert un ticket de déclaration d'anomalie ?

Chaque ticket (ou fiche) de déclaration d'anomalie, indépendamment du gestionnaire d'anomalies utilisé, doit présenter certains attributs indispensables :

1. **Un identifiant** : facilite la traçabilité des défauts
2. **Un résumé** : c'est le **titre** de l'anomalie. Il doit être suffisamment explicite (mots clés). **Le résumé doit permettre à lui seul de comprendre quel est le problème.**
3. **L' impact** : Le niveau de gravité de l'anomalie
4. **La priorité** : Le degré d'urgence à corriger l'anomalie (peut être défini par la MOA)
5. **La reproductibilité** : la capacité à pouvoir reproduire le comportement inattendu
6. **La description précise** de l'anomalie.

NB : Quand l'anomalie est remontée à partir du pas de test, les étapes du test sont récupérées par Squash TM.

En plus de ces éléments indispensables, il est également préconisé de renseigner :

- le **rapporteur**, personne ayant déclaré l'anomalie, ayant créé le ticket d'anomalie;
- la personne ou le service à qui l'anomalie a été **transmise**;
- le **projet** auquel l'anomalie est affectée;
- les **liens** éventuels avec d'autres anomalies;
- le **statut** de l'anomalie (créée, confirmée, en analyse, corrigée, fermée).

**Décrire précisément une anomalie
permet une correction adéquate et rapide**

Le champ « Description » : le premier champ à remplir.

Ce que l'on doit retrouver dans une description :

1. La **configuration** (version du SUT, du navigateur...),
2. Le **jeu de données** et/ou l'**environnement** utilisé (le cas échéant),
3. L'**habilitation** utilisée,
4. Le **composant** concerné (fonctionnel ou technique)
5. L'**objectif** du test
6. La liste des **pas de test** permettant de reproduire l'anomalie.
7. Le comportement **attendu**
8. Le comportement **observé**
9. Des **copies d'écran** de l'anomalie détectée, des **messages d'erreur** (reproductibilité),
10. Des extraits et références aux **spécifications** se rapportant à l'anomalie.

- **Le champ « Résumé » (Titre) :**
- Le résumé doit permettre en quelques mots de comprendre le problème lié à l'anomalie.
- Une nomenclature spécifique (préfixe) doit permettre d'identifier rapidement les éléments de contexte (application, module, macro-fonctionnalité...)
- Éviter les mots parasites.
- **Le résumé doit être clair et auto-porteur**

Ex : *[CONNEXION] Absence de message d'erreur*

- **Le champ « Reproductibilité » :**

reproductible / aléatoire / partiellement reproductible/non reproductible

Il faut pouvoir décrire clairement les manipulations qui ont permis d'aboutir à l'anomalie.

Que faire quand on rencontre une anomalie « non reproductible » ?

- **Documentez les détails** : Prenez note de tous les détails possibles concernant l'anomalie, tels que les étapes précédant l'erreur, les messages d'erreur affichés, les conditions du système, etc.
- **Analysez les journaux** : Consultez les journaux système ou les fichiers de journalisation pour identifier toute information pertinente pouvant aider à comprendre la cause de l'anomalie.
- **Réduisez les variables** : Essayez de simplifier le processus ou le système en éliminant autant de variables que possible. Cela peut aider à isoler la cause de l'anomalie.
- **Expérimentez graduellement** : Essayez de reproduire l'anomalie en introduisant des changements graduels dans le système ou le processus. Cela pourrait révéler ce qui déclenche l'anomalie.
- **Utilisez les outils de diagnostic** : Si possible, utilisez des outils de diagnostic ou de suivi pour surveiller les activités du système en temps réel et identifier les éventuelles anomalies.
- **Collaboration** : Si vous êtes dans un environnement de travail, collaborez avec d'autres membres de l'équipe. Parfois, un regard extérieur peut aider à repérer des éléments que vous auriez pu manquer.
- **Engagez-vous avec la communauté** : Si vous ne trouvez pas de solution interne, recherchez des forums en ligne, des groupes de discussion ou des communautés spécialisées qui pourraient avoir rencontré des problèmes similaires.
- **Testez sur des environnements similaires** : Si possible, essayez de reproduire l'anomalie sur des environnements similaires pour déterminer si elle est spécifique à un système particulier.
- **Sauvegardez les données** : Si l'anomalie a un impact sur les données, assurez-vous de les sauvegarder au préalable, au cas où des actions de résolution pourraient les compromettre.

- **Le champ « Impact » (Gravité, sévérité) : Par ex. critique/ majeure / mineure**
 - Il se mesure en rapport avec le risque métier.
 - La terminologie doit être partagée avec le métier.
 - Une revue régulière avec le métier est nécessaire afin de s'assurer de la bonne sévérité des anomalies.
- **Exemple de niveaux d'Impact (Gravité, sévérité)**
 - **Bloquant** : Anomalie rendant impossible la suite de la recette (plus destiné aux testeurs qu'aux développeurs)
 - **Critique** : Fonction cruciale inopérante pénalisant fortement les utilisateurs sans moyen de contournement possible
 - **Majeur** : Fonction majeure inopérante pénalisant les utilisateurs avec moyen de contournement possible
 - **Moyen** : Fonction importante inopérante pénalisant modérément les utilisateurs avec moyen de contournement possible (Statut rarement utilisé)
 - **Mineur** : Fonction subsidiaire ne pénalisant pas les activités des utilisateurs

- Le champ « **Priorité** » : **faible** / **moyenne** / **forte**
 - La priorité de correction est déterminée ou validée par le *Defect Manager* ou avec l'avis de la MOA
 - Permet de **prioriser la correction d'anomalies** ayant la même sévérité.
- Le champ « **Affecté à** » : A chaque étape de la vie d'une anomalie, la personne en charge de l'anomalie est susceptible de changer.

Affecter un ticket à une personne lui « donne la main » sur le ticket.

Une anomalie non affectée est une anomalie non traitée

- Le champ « **Type** » : Nouvelle anomalie, régression ou carence de spécifications.

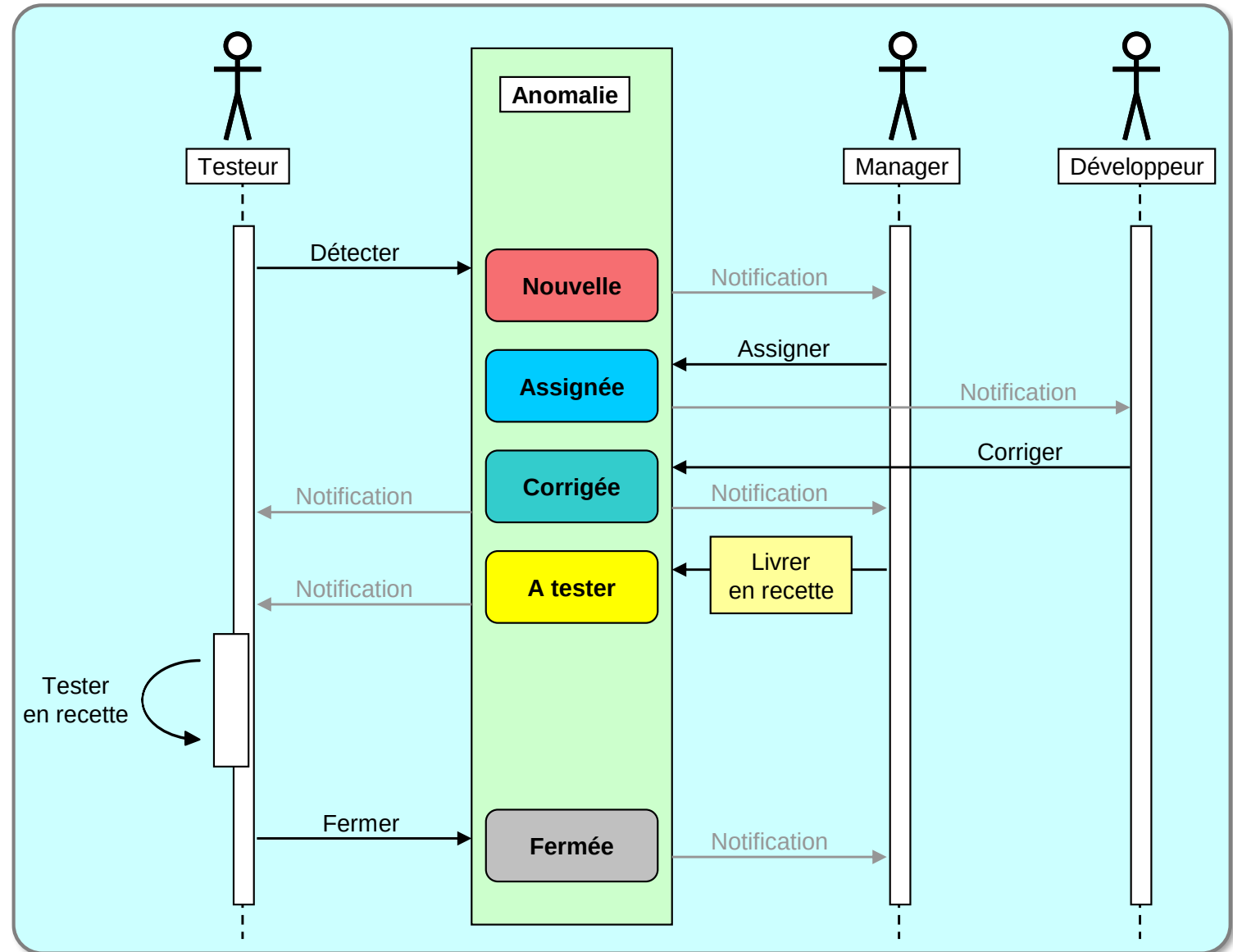
Workflow d'anomalie

Ici, le workflow est représenté par un **diagramme de séquence UML**.*

Y sont représentées les acteurs impliqués dans la « vie » d'une anomalie, leurs actions et les statuts de cette anomalie dans le temps (du haut vers le bas).

Généralement réalisé en phase de Stratégie de recette, ce workflow permet d'établir clairement les responsabilités des acteurs.

* Les diagrammes UML seront étudiés en module 2



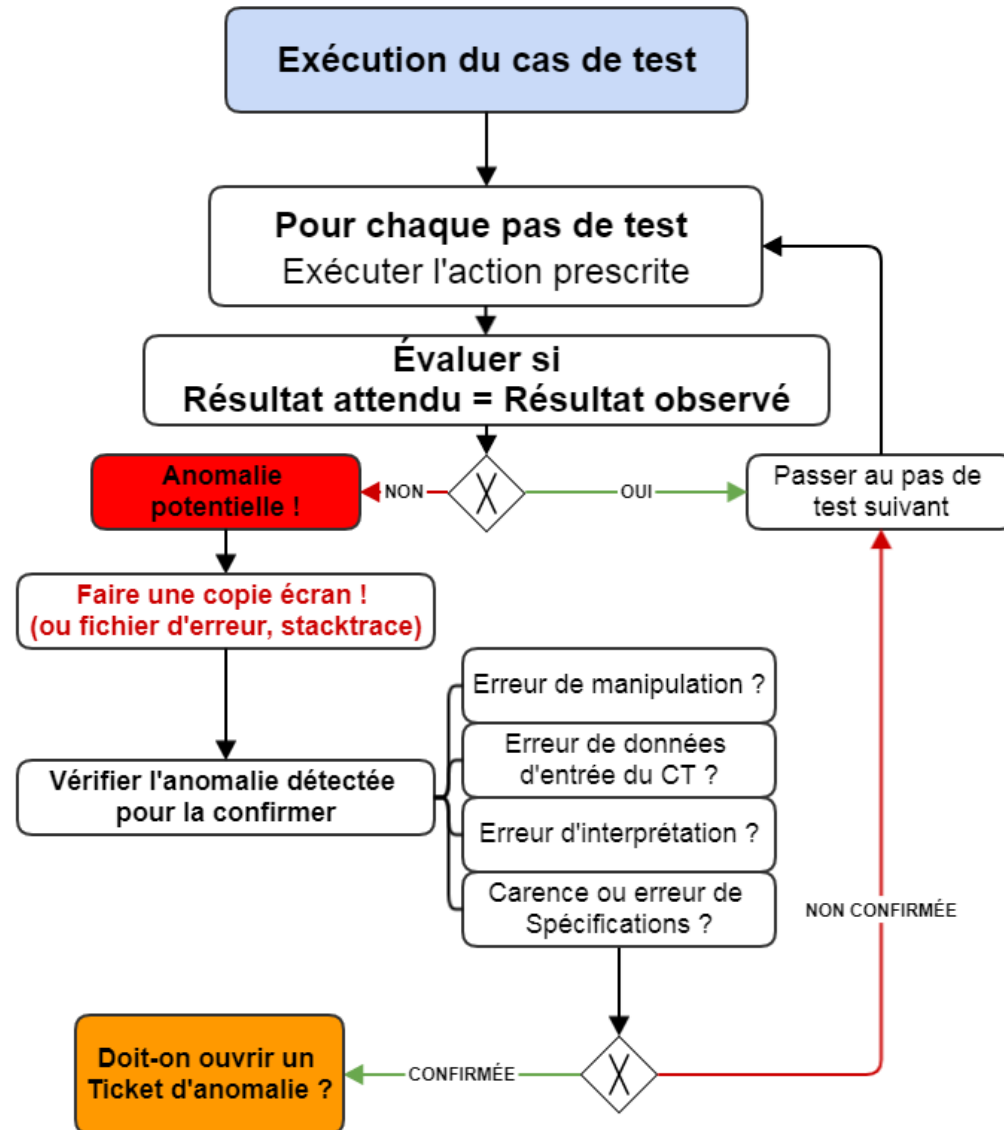
- Dans un cycle/une itération, les testeurs vont **déclarer un certain nombre d'anomalies**.
- Le Defect Manager, s'il existe, va **prioriser** ces anomalies.
- Les équipes de développement vont **corriger** les anomalies prioritaires.
- Sur le cycle suivant/itération suivante, les testeurs vont **vérifier** que les anomalies ont bien été corrigées (*tests de confirmation*).



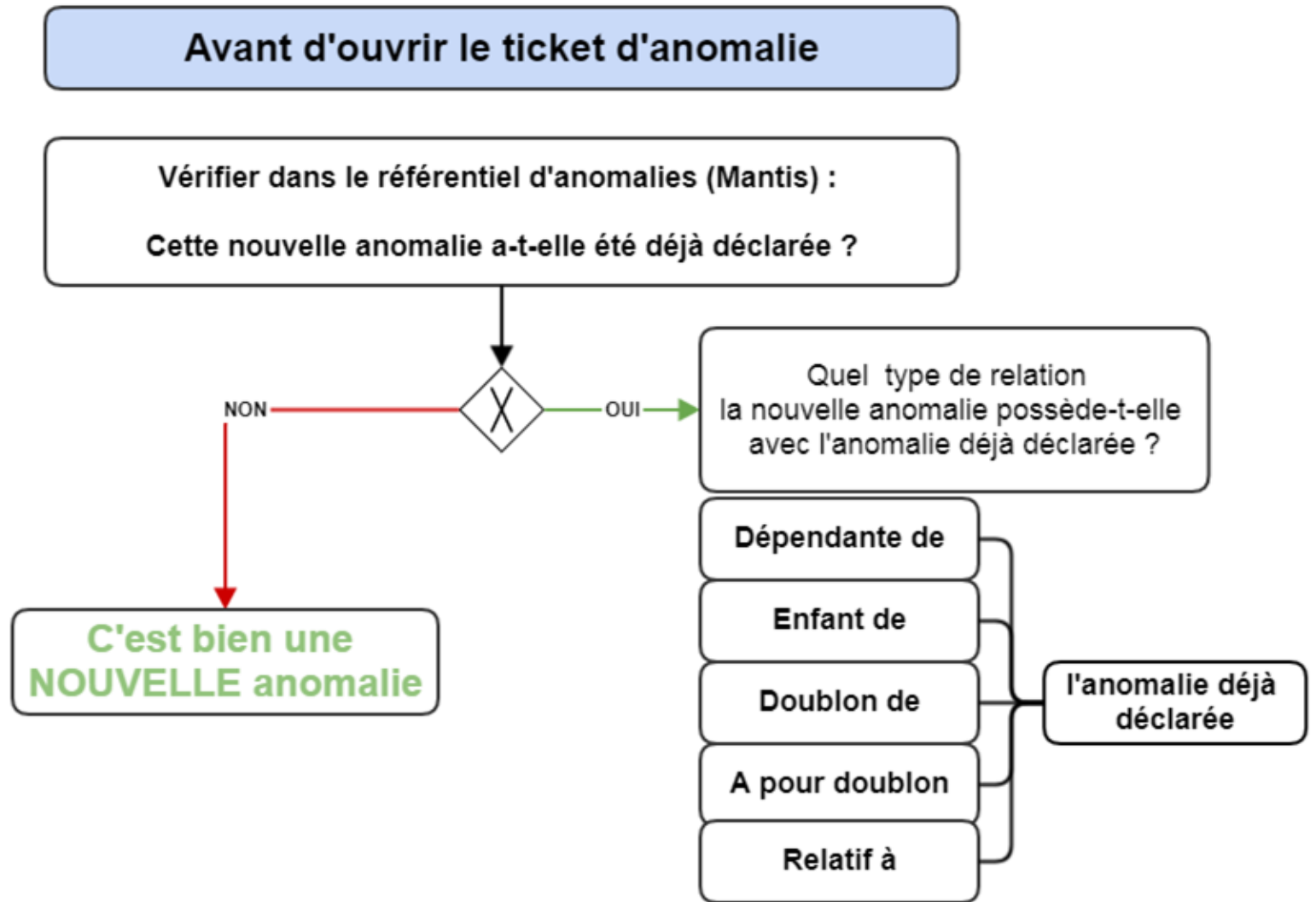
Alternative : Les développeurs peuvent aussi livrer un patch pour permettre de continuer la campagne en attendant la prochaine livraison. Dans ce cas, **l'anomalie reste ouverte** et sa correction sera confirmée **après intégration dans la prochaine livraison**.

- Enfin, à la relivraison, les testeurs vont **effectuer des Tests de Non-Régression (TNR)** afin de s'assurer que les corrections n'ont pas entraîné **d'effets de bord amenant des dégradations** dans les autres fonctionnalités de l'application.

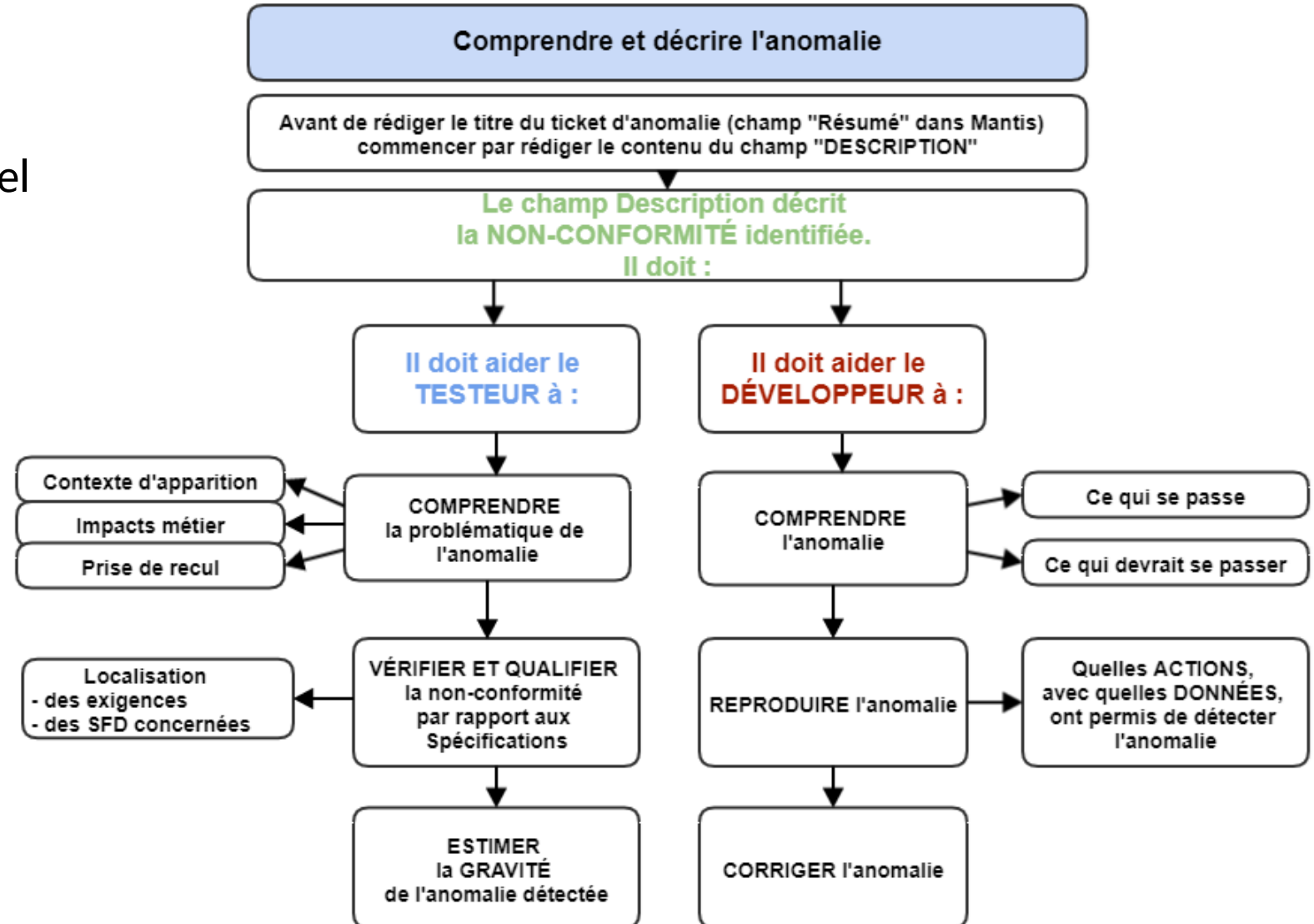
- Détection d'anomalie :
Un processus pour le testeur



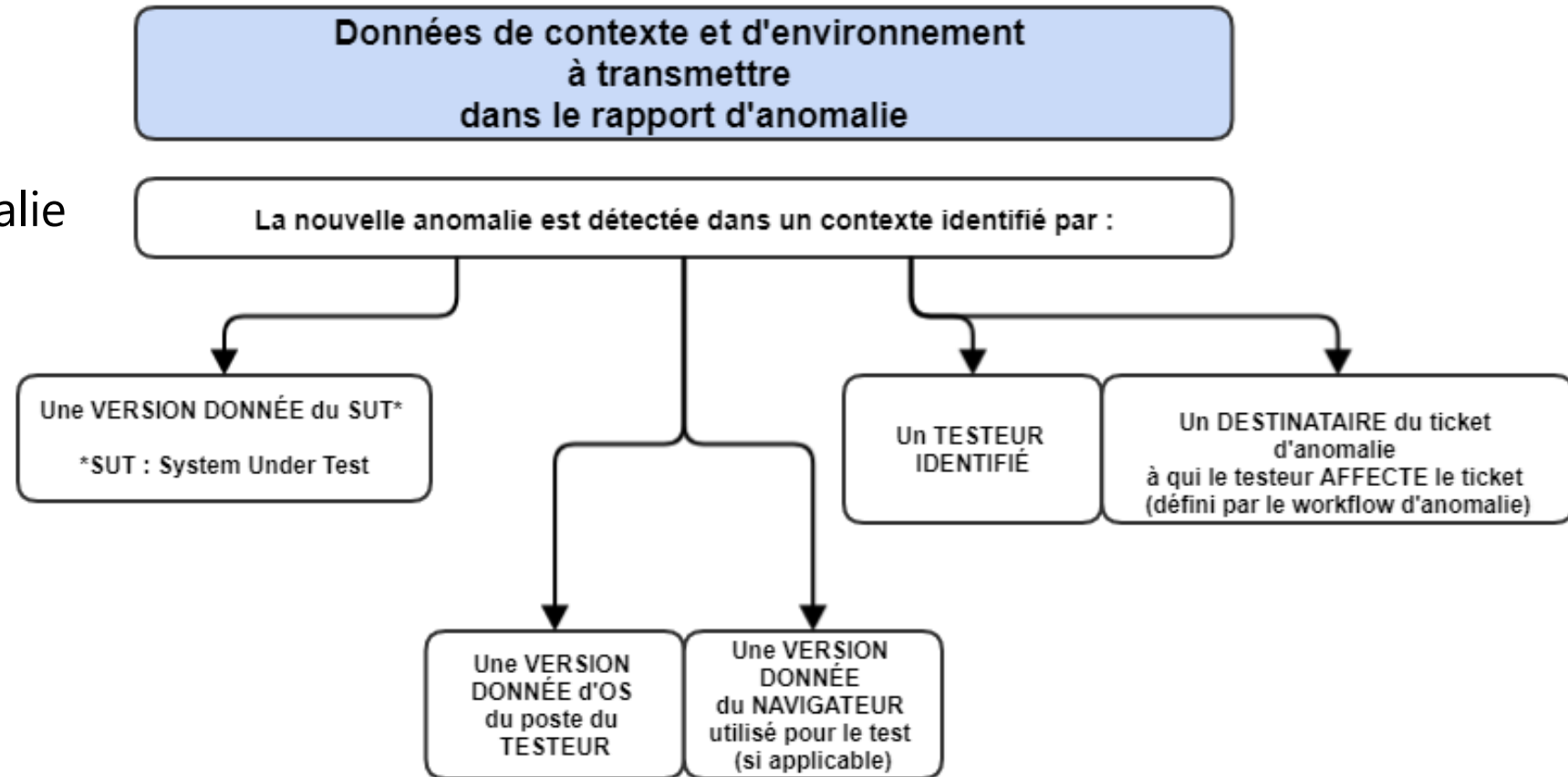
1. Détection d'anomalie
2. Vérification du référentiel d'anomalies (Mantis) :



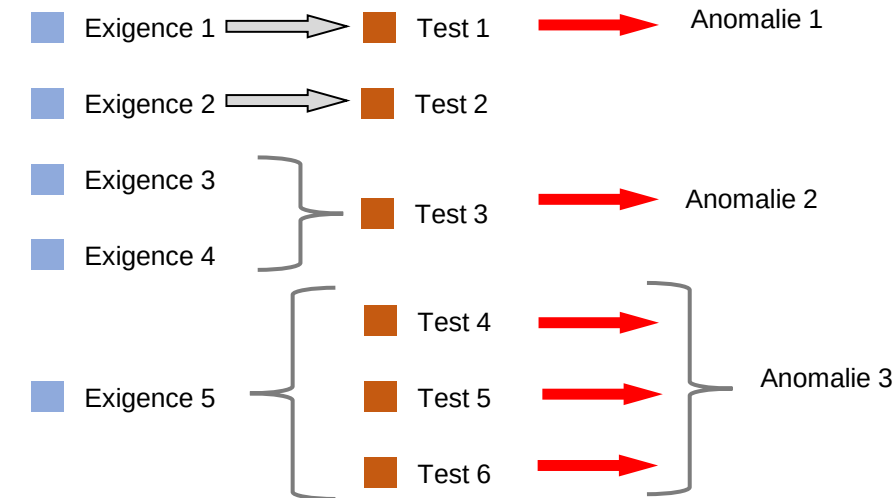
1. Détection d'anomalie
2. Vérification du référentiel d'anomalies (Mantis)
3. Décrire une anomalie :



1. Détection d'anomalie
2. Vérification du référentiel d'anomalies (Mantis)
3. Décrire une anomalie
4. Le contexte de l'anomalie détectée :



- Une même anomalie peut **impacter** plusieurs tests.
- Une même anomalie peut **impliquer** plusieurs exigences.
- Ainsi, dans le cas d'une fonctionnalité qui présente un défaut dans plusieurs cas de figure, il est préférable de créer une seule **anomalie générique** rassemblant tous les cas de divergence.
- **Objectifs :**
 - Simplicité de gestion.
 - Le ticket d'anomalie sert surtout à **communiquer** avec les développeurs.



- Une application en production conserve toujours un **nombre résiduel** d'anomalies.
"Tester prouve la présence d'anomalies, jamais leur absence "
- C'est lors de la Phase de Stratégie de Recette, dans le cadre de la détermination des **conditions de sortie de recette** que la MOA peut fixer le nombre d'anomalies qu'elle jugerait acceptable pour passer en production :
 - Nombre d'anomalies bloquantes
 - Nombre d'anomalies majeures
 - Nombre d'anomalies mineures
- En raison de leurs biais cognitifs propres, les développeurs peuvent parfois accepter difficilement les anomalies remontées. Il faut donc communiquer avec eux de la manière la plus efficace et factuelle possible.

Bien communiquer, c'est :

- Commencer par la **collaboration** plutôt que par la **confrontation**.
- Rappeler à tous **l'objectif commun** d'une meilleure qualité des systèmes.
- Mettre l'accent sur les **bénéfices** du test :
 - Pour les auteurs, l'information sur les défauts peut **les aider à améliorer leurs produits** d'activités et leurs compétences.
 - Pour l'organisation, les **défauts trouvés et corrigés** permettront d'**économiser du temps et de l'argent** et de **réduire le risque global** pour la **qualité du produit**.
- Communiquer les résultats des tests et d'autres constats d'une **manière neutre** et **axée sur les faits, sans critiquer la personne** qui a créé l'item défectueux.
- Rédiger des **rapports objectifs et factuels** sur les défauts et les constats des revues.
- Essayer de **comprendre** ce que **ressent l'autre personne** et les **raisons** pour lesquelles elle peut réagir négativement à l'information.
- Confirmer que l'autre personne **a compris ce qui a été dit** et **vice versa**



Prendre en main un gestionnaire d'anomalie

1. Se loguer sur l'application : <https://squashtm01.ecoleql.fr/mantis/>
2. Utiliser toutes les parties de *MANTIS* : le suivi des anomalies.



Connecter Squash TM et Mantis BT

1. Se loguer sur l'application : <https://squashtm01.ecoleql.fr/mantis/>

2. Cliquer sur « Mon compte » (1) puis sur l'onglet « Jeton API » (2)
3. Choisir un nom explicite (3) et cliquer sur le bouton « Créer » (4)
4. Copier le jeton généré



Connecter Squash TM et Mantis BT

5. Se loguer sur l'application :
<https://squashtm01.ecoleql.fr/squash/>
6. Cliquer sur « Mon compte » (1) puis sur l'onglet « Mode de configuration des Bugtrackers » (2)
7. Choisir le Bugtracker « Mantis » (3)
8. Coller le jeton copier dans Mantis dans le champ Jeton (4)

Cas de test

Campagnes

Pilotage

Automatisation >

Actions <

Administration >

Bugtrackers >

Aide >

FF 1

Mon compte - formateur

Mode de configuration des bugtrackers

Connexion au bugtr... Manuel ☒ Automatique

Enregistrement des ... Mantis (https://squashtm01.ecoleql.fr/mantis) 3

MANTIS (https://squashtm01.ecoleql.fr/mantis)

Jeton 4

Enregistrer Effacer les jetons

Mon compte

Déconnexion



Exécuter la campagne de test pour l'application « Newsletter »

- Créer la campagne
- Ouvrir la page « Newsletter »
- Exécuter les tests
- Remonter les anomalies depuis Squash dans Mantis